

Hooks

Hooks allow you to run custom scripts or shell commands at specific points during Antigravity's execution loop. This is powerful for enforcing custom rules, running linters, or capturing diagnostics automatically.

Configuration

Hooks are configured in a `hooks.json` file located in your customization directory (e.g., `.agents/` in your workspace or `~/.gemini/config/`).

Schema and File Format

The `hooks.json` file maps hook names to their event configurations.

```
json
{
  "my-linter-hook": {
    "PostToolUse": [
      {
        "matcher": "run_command",
        "hooks": [
          {
            "type": "command",
            "command": "./scripts/lint.sh",
            "timeout": 10
          }
        ]
      }
    ]
  },
  "safety-gate": {
    "enabled": false,
    "PreToolUse": [
      {
        "matcher": "run_command",
        "hooks": [
```



```
    ],
  },
  "reminder": {
    "PreInvocation": [
      {
        "type": "command",
        "command": "./scripts/reminder.sh"
      }
    ]
  }
}
```

Hook Definition Fields

Field	Type	Description
<code>enabled</code>	boolean	Optional. Set to <code>false</code> to disable the hook without removing it. Defaults to <code>true</code> .
<code>PreToolUse</code>	array	Handlers that run before a tool is executed.
<code>PostToolUse</code>	array	Handlers that run after a tool completes.
<code>PreInvocation</code>	array	Handlers that run before Antigravity calls the model.
<code>PostInvocation</code>	array	Handlers that run after tool calls finish.
<code>Stop</code>	array	Handlers that run when the execution loop terminates.

Supported Events

		<code>run_command</code>)
<code>PostToolUse</code>	Fires after a tool completes.	Tool name
<code>PreInvocation</code>	Fires before the model is called.	N/A (matcher ignored)
<code>PostInvocation</code>	Fires after tool calls finish.	N/A (matcher ignored)
<code>Stop</code>	Fires when execution terminates.	N/A (matcher ignored)

Matcher

For `PreToolUse` and `PostToolUse`, you can use a regular expression in the `matcher` field to specify which tools trigger the hook:

- `""` or `"*"` : Match all tools.
- `"run_command"` : Match exactly `run_command`.
- `"run_command|view_file"` : Match either tool.
- `"browser_.*"` : Match any tool starting with `browser_`.

① **Note:** For `PreInvocation`, `PostInvocation`, and `Stop`, the structure is simpler (a list of handlers directly under the event key) and the matcher is ignored.

Supported Tools

For `PreToolUse` and `PostToolUse` matchers, you can match against the following tool names, grouped by category:

File and Directory Operations

- `view_file` : View the contents of a file.
 - Arguments: `AbsolutePath`, `StartLine` (optional), `EndLine` (optional), `IsSkillFile` (optional)

- Arguments: `TargetFile` , `Instruction` , `Description` , `AllowMultiple` , `TargetContent` , `ReplacementContent` , `StartLine` , `EndLine` , `TargetLintErrorIds` (optional)
- `multi_replace_file_content` : Make multiple, non-contiguous edits to the same file.
 - Arguments: `TargetFile` , `Instruction` , `Description` , `ReplacementChunks` (array of chunks), `TargetLintErrorIds` (optional), `ArtifactMetadata` (optional)
- `list_dir` : List the contents of a directory.
 - Arguments: `DirectoryPath`
- `find_by_name` : Search for files and directories using glob patterns.
 - Arguments: `SearchDirectory` , `Pattern` , `Type` (optional), `Excludes` (optional), `Extensions` (optional), `FullPath` (optional), `MaxDepth` (optional)

Search and Research

- `grep_search` : Fast text searches within specific paths.
 - Arguments: `SearchPath` , `Query` , `IsRegex` (optional), `CaseInsensitive` (optional), `Includes` (optional), `MatchPerLine` (optional)
- `search_web` : Perform a general web search.
 - Arguments: `query` , `domain` (optional)
- `read_url_content` : Fetch text content of a public URL.
 - Arguments: `Url`

System and Execution

- `run_command` : Propose a bash command to run.
 - Arguments: `CommandLine` , `Cwd` , `WaitMsBeforeAsync` , `RunPersistent` (optional), `RequestedTerminalID` (optional)
- `manage_task` : Interact with background tasks.
 - Arguments: `Action` (`'list'` , `'kill'` , `'status'` , `'send_input'`), `TaskId` (optional), `Input` (optional)
- `schedule` : Set timers or recurring cron jobs.
 - Arguments: `DurationSeconds` (optional), `CronExpression` (optional), `MaxIterations` (optional), `Prompt`

Arguments: `reason`, `target`, `reason`

Agent Collaboration

- `invoke_subagent` : Spawn specialized sub-agents.
 - Arguments: `Subagents` (array of specs with `Prompt`, `Role`, `TypeName`, `Workspace` (optional))
- `define_subagent` : Create a custom sub-agent.
 - Arguments: `name`, `description`, `system_prompt`, `enable_mcp_tools` (optional), `enable_write_tools` (optional), `enable_subagent_tools` (optional)
- `send_message` : Communicate with other agents.
 - Arguments: `Recipient`, `Message`
- `manage_subagents` : List or terminate active sub-agents.
 - Arguments: `Action` (`'list'`, `'kill'`, `'kill_all'`), `ConversationIds` (optional)

Interaction and Media

- `ask_question` : Ask multiple-choice questions.
 - Arguments: `questions` (array of questions with `question`, `options`, `is_multi_select`)
- `generate_image` : Create or edit images.
 - Arguments: `Prompt`, `ImageName`, `ImagePaths` (optional)

Hook Handler Configuration

Each item in the `hooks` array supports:

Field	Type	Description
<code>type</code>	string	Optional. Currently only <code>"command"</code> is supported. Defaults to <code>"command"</code> .
<code>command</code>	string	Required. The shell command to execute.

Input/Output Contract

Hooks receive input via **stdin** as JSON and should return output via **stdout** as JSON. Field names use camelCase.

Common Input Fields

All hooks receive the following system metadata fields in their input payload on `stdin` :

Field	Type	Description
<code>conversationId</code>	string	The unique UUID of the active agent conversation
<code>workspacePaths</code>	array of strings	Absolute directory paths representing the user's r
<code>transcriptPath</code>	string	The absolute path to the persistent <code>transcript.j</code> Note: This file lives in: <code><app_data_dir>/brain/<conversationId>/ .system</code> where <code><app_data_dir></code> is: <code>~/.gemini/antigravity</code> for Antigravity 2.0 <code>~/.gemini/antigravity-cli</code> for CLI
<code>artifactDirectoryPath</code>	string	The absolute path to the directory containing all c screenshots.

PreToolUse

Fires before a tool is executed.

Schema

<code>toolCall</code>	object	Details of the proposed tool call.
<code>toolCall.name</code>	string	The name of the tool being executed (e.g., <code>run_command</code>).
<code>toolCall.args</code>	object	The arguments passed to the tool.
<code>stepIdx</code>	integer	The 0-based index of the current step in the trajectory.
<i>(Common Fields)</i>	Includes <code>conversationId</code> , <code>workspacePaths</code> , <code>transcriptPath</code> , <code>artifactDirectoryPath</code> .	

Output Fields (stdout):

Field	Type	Description
<code>decision</code>	string	Required. Controls how the tool call is gated: - <code>"allow"</code>: Automatically allows the tool execution. - <code>"deny"</code>: Hard blocks execution immediately. - <code>"ask"</code>: Prompts the user, but respects "Always Allow" settings. - <code>"force_ask"</code>: Always prompts the user, ignoring cached permissions.
<code>reason</code>	string	Optional. The explanation shown to the agent or user for the decision.

strings

override default tool permissions.

Example

- **Input (stdin):**

```
json
{
  "toolCall": {
    "name": "run_command",
    "args": {
      "CommandLine": "npm test",
      "Cwd": "/workspace/project",
      "WaitMsBeforeAsync": 5000
    }
  },
  "stepIdx": 19,
  "conversationId": "ec33ebf9-0cba-4100-8142-c61503f6c587",
  "workspacePaths": [
    "/workspace/project"
  ],
  "transcriptPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587/.system_generated/logs/transcript.jsonl",
  "artifactDirectoryPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587"
}
```

- **Output (stdout):**

json

```
    // comment (npm - 0000),
  ]
}
```

PostToolUse

Fires after a tool completes.

Schema

Input Fields (stdin):

Field	Type	Description
<code>stepIdx</code>	integer	The 0-based index of the completed step.
<code>error</code>	string	Optional. The detailed runtime error message if the tool call failed. Empty if successful.
<i>(Common Fields)</i>	Includes <code>conversationId</code> , <code>workspacePaths</code> , <code>transcriptPath</code> , <code>artifactDirectoryPath</code> .	

Output Fields (stdout): Returns an empty JSON object `{}`.

Example

- **Input (stdin):**

```
json
{
  "stepIdx": 5,
  "error": "exit status 1",
```



```
      "workspacePath": "~/.gemini/antigravity/brain/ec33ebf9-8142-c61503f6c587/.system_generated/logs/transcript.jsonl",
      "artifactDirectoryPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587"
    }
  }
```

- **Output (stdout):** `{}`

PreInvocation

Fires before the model is called.

Schema

Input Fields (stdin):

Field	Type	Description
<code>invocationNum</code>	integer	The 0-indexed sequence number of the current model invocation (the first invocation is 0).
<code>initialNumSteps</code>	integer	The number of steps currently in the trajectory.
<i>(Common Fields)</i>	Includes <code>conversationId</code> , <code>workspacePaths</code> , <code>transcriptPath</code> , <code>artifactDirectoryPath</code> .	

Output Fields (stdout):

objects trajectory before the model is called.

Injected Step Schema: Each object in the `injectSteps` array can have one of the following fields:

- `toolCall` (object): A tool call to execute.
- `userMessage` (string): A message from the user.
- `ephemeralMessage` (string): A transient system message.

Example

- **Input (stdin):**

```
json
{
  "invocationNum": 3,
  "initialNumSteps": 10,
  "conversationId": "ec33ebf9-0cba-4100-8142-c61503f6c587",
  "workspacePaths": [
    "/workspace/project"
  ],
  "transcriptPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587/.system_generated/logs/transcript.jsonl",
  "artifactDirectoryPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587"
}
```

- **Output (stdout):**

```
json
{
  "injectSteps": [
    {
      "ephemeralMessage": "Remember to lint"
    }
  ]
}
```

PostInvocation

Fires after tool calls finish.

Schema

Input Fields (stdin): Same as `PreInvocation` input fields.

Output Fields (stdout):

Field	Type	Description
<code>injectSteps</code>	array of objects	Optional. List of steps to inject after the invocation completes (same schema as <code>PreInvocation</code> inject steps).
<code>terminationBehavior</code>	string	Optional. Controls the execution flow after injection: - <code>"force_continue"</code>: Forces the loop to continue. - <code>"terminate"</code>: Forces the loop to terminate. - <code>""</code> (or omitted): Default behavior.

Example

- **Input (stdin):** Same as `PreInvocation`
- **Output (stdout):**

```
json
{
  "injectSteps": [],
  "terminationBehavior": ""
}
```



Input Fields (stdin):

Field	Type	Description
<code>executionNum</code>	integer	The sequence number of the execution attempt.
<code>terminationReason</code>	string	The reason why the execution is stopping (e.g., <code>"model_stop"</code> , <code>"max_steps_exceeded"</code> , <code>"error"</code>).
<code>error</code>	string	Optional. The error message if termination was caused by a system error.
<code>fullyIdle</code>	boolean	Required. <code>true</code> if the agent is completely finished and all background commands or asynchronous tasks have completed. <code>false</code> if active background tasks are still running.
<i>(Common Fields)</i>	Includes <code>conversationId</code> , <code>workspacePaths</code> , <code>transcriptPath</code> , <code>artifactDirectoryPath</code> .	

Output Fields (stdout):

Field	Type	Description
<code>decision</code>	string	Required. Set to <code>"continue"</code> to prevent the agent from stopping and re-enter the execution loop. Any other value

reason

string

Optional. If `decision` is `"continue"`, this message is injected as a system message into the conversation.

Example

- **Input (stdin):**

```
json
{
  "executionNum": 1,
  "terminationReason": "model_stop",
  "error": "",
  "fullyIdle": true,
  "conversationId": "ec33ebf9-0cba-4100-8142-c61503f6c587",
  "workspacePaths": [
    "/workspace/project"
  ],
  "transcriptPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587/.system_generated/logs/transcript.jsonl",
  "artifactDirectoryPath": "~/.gemini/antigravity/brain/ec33ebf9-0cba-4100-8142-c61503f6c587"
}
```

- **Output (stdout):**

```
json
{
  "decision": "continue",
  "reason": "Not done yet"
}
```

